Linnéuniversitetet
Kalmar Växjö

Project in Embedded Systems

Project Drone

Authors:
Noa Cluer, Samuel Andersson
Course Responsible:
Hemant GHAYVAT
Semester: VT26
Course code: 2DT304

Abstract

Project Drone uncovers the optimization of cost efficiency, design simplicity, carrying capacity and flight stability of drones, through an inductive experimental research process of iteratively designing and constructing a drone. Drones, or more specifically quadcopter micro-drones, see an ever increasing usage in both professional, scientific and private fields, ranging from delivery of healthcare products, to hobby photography, and agricultural applications. With many external disturbances and complex internal designs, the drone design field is broad, and the development complex by nature. Due to the variety of application areas, the concept of a universally perfect drone is impossible, requiring ever ongoing research to continue optimization. Our approach to this issue is to design a drone from the ground up, using available components and techniques, while also minimizing component costs, yet without loosing usability. This report provides instructions on how to make such a drone while documenting the iterative development process, and provides an analysis in comparison to an established market-available option, giving valuable insights into optimization of the mentioned metrics.

Keywords: *Quadcopter, UAV, Quadrotor, Embedded System, IMU, ESC, μUAV, micro-drone, Crazyflie 2.1+, Raspberry Pi Pico W, flight controller, CAD, drone development, micropython, FDM printing, thrust, aerodynamics*

Contents

List of Figures

1 Introduction **1**
- 1.1 Introduction / Background . . . 2
- 1.2 Problem definition . . . 4
- 1.3 Purpose and research question / hypothesis . . . 5
- 1.4 Scope / Limitation . . . 6
- 1.5 Target group . . . 7
- 1.6 Outline . . . 8

2 Related Work **9**
- 2.1 Commercially available similar products . . . 9
- 2.2 Previous academical knowledge . . . 10

3 Hardware **11**
- 3.1 Embedded board . . . 11
- 3.2 Output devices . . . 12
- 3.3 Drone body . . . 13
- 3.4 Component table & Circuit diagrams . . . 14
- 3.5 Power . . . 15

4 Software **17**
- 4.1 Phase 1 software (thrust testing) . . . 17
- 4.2 Phase 2 software (navigation and stabilization) . . . 18
- 4.3 Programming language . . . 20
- 4.4 Security threats . . . 20

5 Results and Discussion **21**
- 5.1 What are phases? . . . 21
- 5.2 Initial system design . . . 23
- 5.3 Drone prototype: Alpha series . . . 24
- 5.4 Drone prototype: Beta series . . . 25
- 5.5 Metric optimization . . . 26
- 5.6 Drone prototype: Gamma series . . . 27
- 5.7 Drone prototype: Delta series . . . 28
- 5.8 Evaluation of metrics . . . 30

6 Conclusion **32**

References **35**

A main.py (updated version of drv8833_test.py) **1**

B drv8833_test.py (beta prototype series version) **4**

C Delta v4.1 liftoff test **7**

D Timelog **9**

E Component list for delta prototype **12**

List of Figures

1.1 Proposed classification of drones as per [1] 2
1.2 A typical Quadcopter system layout 4
1.3 Project Scope Diagram 7
2.4 The *Crazyflie 2.1+* 9
3.5 The *Raspberry Pi Pico W* board 11
3.7 The parts of a drone chassis *(prototype 1.3)* 13
3.8 Wiring diagram for drone prototype 14
3.9 Circuit diagram for drone prototype 15
3.10 A LiPo battery 15
4.11 A drone's movements 18
5.12 Project timeline graph 22
5.13 The *MPU6050* pinout 23
5.14 Primary motor test 23
5.15 The *Alpha* chassis prototype series 24
5.16 The *Beta* chassis prototype series 25
5.17 The assembled *Beta* prototype 26
5.18 The *Gamma* chassis prototype series 27
5.19 The assembled *Gamma* prototype 28
5.20 The assembled *Delta v4.1* prototype 29
5.21 The assembled, non-functional, *Delta v4.2* prototype 30

1 Introduction

This section presents a detailed overview over the project's research background, justification or problem description, objective and goals, project scope, and target audience.

The class of *lightweight unmanned air vehicles (UAVs)* known as micro drones, are a rapidly evolving tool for navigating and efficiently performing tasks in environments that are inaccessible, dangerous, or otherwise hostile to human inspection [1]. Referring to all drones in the weight span of ≤ 5kg, the development of micro drones is under heavy resource constraints due to the limitations in total weight [2]. This limitation in performance to size, ensures micro drones stay more agile than their larger counterparts, and allows for travel in confined spaces otherwise inaccessible. The agility and versatility of micro drones makes them desired in multitudes of fields including intelligence collection, package delivery, agriculture, and even security and military defense [3]. The total micro drone market size has been estimated at USD $ 5,27 billion [4].

With the ever evolving and expanding drone industry, there is a constant demand to increase carrying capacity and flight stability, while simultaneously decreasing production cost and general complexity for wider application potential. *"Generally, to design fixed wing drones, researchers used trial and error methods which increase the cost and time of the design process [242–244]. Because of using trial and error methods in designing fixed wing drones, their design CANNOT be considered optimized due to uncertainties in weight estimation, sizing, selection of the best wing shape and aspect ratio for maximum endurance."* [1].

Bringing the above methodology into the *quadrotor* development field, this research paper will attempt contribution, by analyzing the optimal balance between carrying capacity, flight stability, production cost and system complexity. The study will be conducted through experimental research, analyzing how key components and design decisions impact these performance metrics. Analysis will be continuously performed, on micro drone prototypes whilst developing our own optimally balanced drone.

1.1 Introduction / Background

Drones have a wide area of application, being developed for a variety of fields, each with their own requirements and conditions [5]. Multiple main types have therefore been developed, each for their own application area, however this essay will only mention quadcopters *(from here on sometimes referred to simply as drones)*. For those further interested we would refer to the chart in *Figure 1.1*. The type that we will be working on is a *Micro Unmanned Aerial Vehicule (μUAV)*, with *Vertical Take off and Landing (VTOL)*, and a *Quadcopter* layout *(Quadrotor)*. Drones are used in a number of fields including but not limited to: intel for defence purposes, delivery of medical aid packages in remote areas, damage evaluations in disaster areas, routine inspections of places difficult to access, and even agriculture [5]. The field of drone development is therefore a broad term for optimization and development of drones suitable and optimized for, their specific intended use-cases.

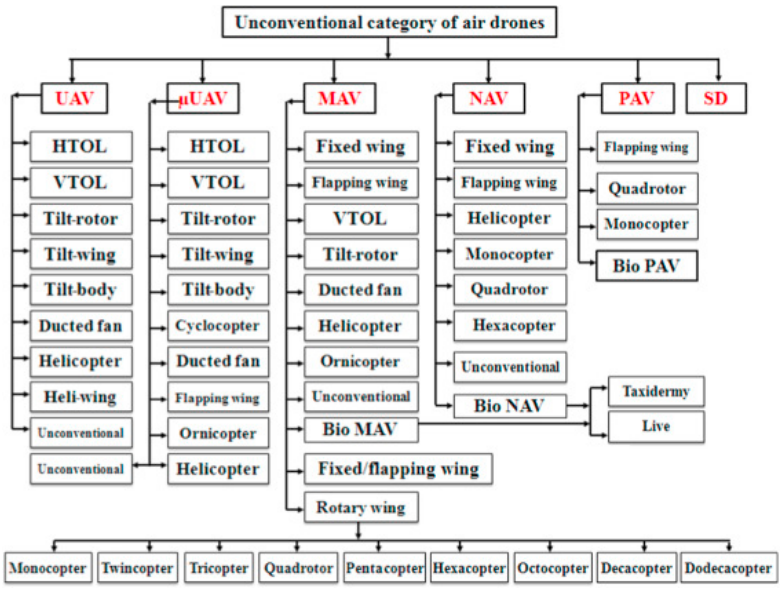


Figure 1.1: Proposed classification of drones as per [1]

"An embedded system is a microprocessor based system that is built to control a function or a range of functions." [6]. A *Quadcopter* typically has a designated function; it could be providing aerial view of an area, photography, or any number of other activities [5, 7]. The internal functionality of a drone would classify as complex enough to be referred to as a system, and it is controlled by a *Flight Controller*, typically a *Micro Control Unit (MCU)*, centered around a *micro processor*. This argument is further

supported by drones consisting of a central processing unit *(said flight controller)* receiving sensor feedback from an *Inertial Measurement Unit (IMU)*, or other navigational feedback device. That sensor feedback is then utilized by the drones' actuators (propellers) to keep flight stable. On top of this embedded on-device functionality, there is typically a real-time wireless connection back to a human pilot, providing relevant data, and receiving back new flight instructions [7].

Today, the usage of *micro-drones* of (typically) *quadcopter* design, is a quickly expanding field used by a variety of companies [8]. One after the other, industries previously unknown to drone usage are provided with the agility, flexibility and intelligence gathering capabilities of *µUAV's* [9, 10, 11, 12, 13]. Even on the battlefields of the Ukrainian invasion, the flexibility of *micro drones* has taken focus away from more traditional larger warfare drones [14]. Considering this ever increasing relevancy and market expansion, the continued research on *µUAV* development is necessary for meeting the optimization and performance requirements of current market demand.

Structurally, a typical *quadcopter* consists of [7-9] major subsystems (depending on how they are grouped or soldered), interacting in accordance to *Figure 1.2*; A pilot handling a ***controller*** sends navigation instructions to the drone, and examining the sensor feed (if present). Communication is oftentimes upheld over *Low-Frequency Radio communication* sent to the drones' radio-***communication*** receiver. The entire drone device is powered by a singular ***battery*** providing current to each component. Power to the individual ***motors*** is regulated through *Electronic Speed Controllers (**ESC's**)*, a crucial aspect for generating the resulting force vectors required to change the drones direction and elevation (as elaborated on in section 2). The specific throttling of each motor is modulated by the ***Flight Controller*** (often a *micro controller*), based on navigation instructions from the pilot, and current positional data provided by the *Inertial Measurement Unit (**IMU**)* (or other balancing device). The *IMU* provides the measurements required to avoid navigational drift, and ensure stability in hovering. There can also be ***sensors*** on the drone; for example *First Person View (FPV) drones* have a live video feedback to the pilot to ease navigation [15]. All of the subsystems are then packaged together inside of a ***chassis*** (also known as a frame) to allow for a solid structure protecting components during flight.

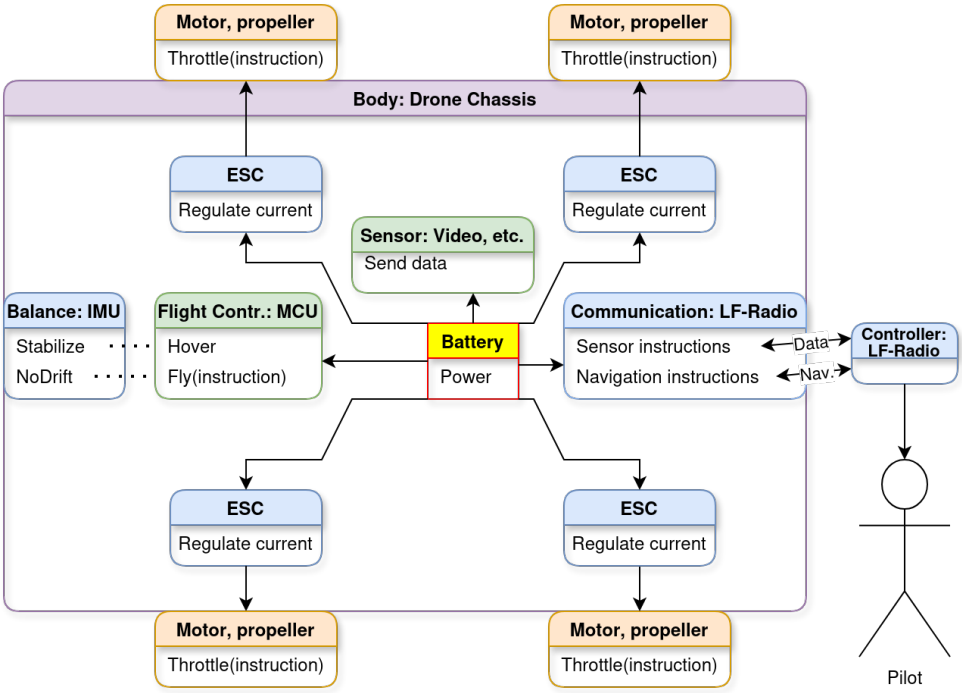


Figure 1.2: A typical Quadcopter system layout

1.2 Problem definition

As aforementioned, optimization of a drone to perfection, is impossible by the nature of the development process [1]. Several design challenges including hardware selection, software structure and their interactions introduce variability into systems. *"The control of these factors determines various performance metrics including flight endurance, operational parameters, and payload capabilities."* [16]. When developing drones based on trial and error, it is important to keep experimenting and redesigning past an established successful design. This way, previously untested (and potentially more efficient) designs can be evaluated and documented, providing additional knowledge to the field. *"Small drones (μUAV, MAV, NAV, and PAV), are not merely scaled down versions of larger airplanes. Since all the characteristics of larger airplanes have to be retained in a small volume, the challenges and complexity in their design and fabrication increase significantly."*, and also *"In current design theories of drones, the ability to transform and change the configuration can be considered as a new field of research."* [1]. To optimize a drone design to the fullest achievable degree, allows for energy and material savings [17]. Considering the current climate issues drones provide the potential for energy-efficient transport of goods, while also having

the ability to easily gather live data from areas difficult to access with conventional infrastructure or vehicles. Both of these abilities prove invaluable during search and rescue operations, and disaster assessments. Access to more and better environmental data would also allow the people in power to make better informed decisions regarding climate matters [17].

Micro UAV drones are under constant development, shrinking and adding functionality and performance with each iterative model [8]. For the trend to continue this way without shutting out smaller companies and private actors from the development sector, the designs need to be more cost effective and simplified to be more accessible to the general public. Making projects accessible to smaller market actors tends to have positive results on development and productivity [18], and it would make access to simpler but commercially capable drones easier for many across the world that are living in rural areas [19]. Additional benefits for usage in rural areas is that drones do not require a maintained road system, a common issue in parts of the world, preventing the use of larger land-bound vehicles. By simplifying the manufacturing process, the number of failure points is reduced *"the fabrication process is one of the important steps in the creation of drones. In fact, every step in the manufacturing process affects the final performance of the drone. Thus, in the manufacturing stage of drones, it is important to determine the manufacturing method and the used material to fabricate them."* [1].

"The price of commercially available drones ranges from $30 to $300,000. It is difficult for recreational, commercial, and public safety users to determine which drone to select based on their needs. It is vague if the price of the drones is impacted by certain specifications or is it all decisions of the sellers without any basis." [20]. Considering these predetermined conditions, it is difficult to assess a target-cost, since this cost-range includes UAVs with cutting edge technology to drones that are merely toys. By exploring the decreasing of production costs and simplification of the design it is possible to achieve commercial performance levels, while maintaining the costs of a toy drone, increasing accessibility to low capital market actors.

1.3 Purpose and research question / hypothesis

The main research question is *inductively* analyzing how different parameters and design decisions impact the performance of a micro drone, when optimizing for cost effectiveness, design simplicity, carrying capacity and stability.

The research will be conducted using *experimental research* in an iterative process involving two phases. **Phase 1** will consist of designing and constructing an initial quadcopter prototype, following the construction of

the completed prototype, it will undergo a flight and carry test where significant performance data will be collected. Finally the design will be evaluated. Any valuable insights gained during the phase will be documented.

The research will then transition to **Phase 2**, where the insights collected will be applied to design and construct a new quadcopter configuration. The new configuration will then undergo an identical flight performance test and its result will be compared and evaluated to the previous design. Phase 2 will be repeated until a carrying capacity large enough for stable flight with micro-load sensors on board has been achieved, or until the end of the research period.

The results to be achieved at the end of the research period are: a functional quadcopter drone in the micro class optimized for cost effectiveness, design simplicity, stability and carrying capacity, with full documentation covering detailed schematics, step by step construction process, source code and hardware lists. The report will also cover a comparative analysis between the development prototypes, and commercially available alternatives, thereby adding *explanatory knowledge* to the field.

1.4 Scope / Limitation

When the project specification *(figure 1.3)* was planed out, the research area of *drones* came to mind due to their aforementioned relevancy. By developing a drone in-house, instead of working with application of a prebuilt, the expected outcome was to gain considerably deeper insights into the development field, simultaneously contributing research to it. The drone classes of reasonable scale considering our development resources were micro drones (μUAVs) or nano drones (NAV), where a *μUAV* was opted for, to have the freedom of working on a slightly larger scale, and utilize standard laboratory components. NAVs are typically small [1], to a degree where they would be difficult for us to develop without specialized custom made equipment. Though hexacopters are not uncommon, the norm is usage of *quadcopter* layouts, and the addition of two motors would introduce extra complexity and size to the model. A number of metrics *(cost, complexity, stability and carrying capacity)* were selected as focus four optimization, with flight duration being skipped due to relying on the battery used and due to time constraints.

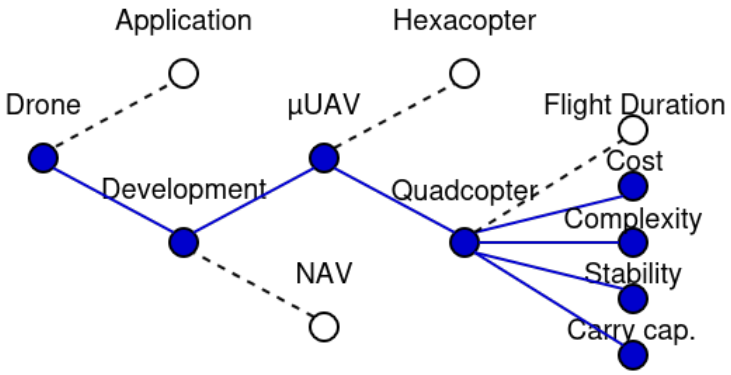


Figure 1.3: Project Scope Diagram

The drone will not be tested under any extreme conditions such as extreme temperature, wind, rain, etc., or by any third party users. The reason for this is time and budget constraints.

1.5 Target group

The research primarily aims to target the developing and expanding drone industry by giving valuable insights in the development of micro drones including optimization of target parameters and costs. These insights can also be of value for open-source hardware communities or resource-constrained prototyping.

1.6 Outline

Introduction: The introduction presents a detailed overview over the project's research background, justification or problem description, objective and goals, project scope, and target audience. The introduction is beneficial for all who are not already well versed in the field of drone development. Do however note that section 1.3 contains our research question, and is therefore something that everyone should read.

Hardware: Presents the hardware components utilized in the drone prototyping process, detailing their exact specifications, functionality, pinout schematics and overall system design. The hardware chapter is beneficial primarily for beginners in electronics and drone development, as most components are common and widely used.

Software: Explains the theoretical structure and features of the developed code, and elaborates upon the firmware and chosen programming language. The software chapter is beneficial especially when expanding further upon this project, or looking to understand the stabilization and navigation of a drone.

Results & Discussion: Describe the process of initial system design, the project phases, and the iterative drone prototyping process. Accompanying the results are discussions of what improvements and goals are achieved, and notes about encountered errors and problems. It is also in the results & discussion that you will find final material costs, and descriptions of completed tests for our optimization metrics. This section is recommended for everyone looking for more than a brief overview, since it has the analysis that is a core part of this report.

Conclusion: Presents how work was distributed within the group, and elaborates more upon the encountered challenges. It also contains tips for anyone looking to work further on this project, especially in the form of hardware recommendations.

For a **minimal overview**, we would recommend examining figure 1.3 for the research question, and figure 1.2 for a brief background into the main components of a drone. The main analysis is in section 5.8, with overall project discussion in section 6.

2 Related Work

Micro drones are a popular area, and their usage in various fields is still increasing. Therefore there is a lot of research and previously written material on the general area, and this section will present a couple of the most relevant ones.

2.1 Commercially available similar products

Available for purchase there is the *"Crazyflie 2.1+"* (figure 2.4), a μUAV replacing the older and now discontinued *"Crazyflie 2.1"* and *"Crazyflie 2.0"* [21, 22, 23]. Designed with focus on ease of implementation and ease of modification, it is intended for education, research and drone swarms, and all of its software and firmware is open-sourced. It is equipped with *4 $\times$ 7mm DC coreless motors*, a *250mAh 15C discharge* battery, an embedded 3-axis gyroscope/accelerometer IMU *BMI088*, a pressure sensor *BMP388*, and pins supporting I^2C, PWM, SPI and more. The flight controller consists of the the IMU, the pressure sensor, a *nRF51822* radio and bluetooth chip (with its own MCU), and a main MCU; *STM32F405* with an M4 Cortex, 168MHz clock speed, 192kb SRAM, and 1Mb flash memory. The Crazyflie 2.1+ has an estimated flight time of 7 minutes, a maximum recommended payload of 15g [24], and costs $300 (including VAT) [25]. The largest difference between the Crazyflie and our prototype will be our cost effectiveness metric, and our use of more widely available, simpler components. Attempting to reach similar properties overall *(simplicity, availability and system scale)*, we will focus on greatly reducing the costs of finished prototypes, and will also attempt to achieve greater safe carrying capacity than the 15g of the Crazyflie 2.1+.

Figure 2.4: The *Crazyflie 2.1+*

2.2 Previous academical knowledge

"Design And Development of Flight Controller For Quadcopter Drone Control" [26] is a thesis written in 2024 where the researchers also explored the usage of the *MPU6050* as *IMU* for a small quadcopter. The main difference being that they focused on the stabilization aspects, and general functionality of the *flight controller*. In this report, we also worked on developing our own drone design from scratch, allowing us to focus on the cost effectiveness, design simplicity and carrying capacity on top of working with stabilization. Having come to very similar conclusions about stabilization, we can validate our results against theirs, and from a stable knowledge basis continue our optimization of the other metrics.

"Energy Consumption Modeling and Flight Time Analysis of Micro Drones" is another work with similar μUAVs, published in mid 2025 [3]. Therein the authors, rather than optimizing their own drone design, explore the limits of the battery on bought ones. They provide some background information about UAVs also found in this report, and elaborate further on the expected behavior of µUAVs at lower battery levels, a relevant topic when considering energy efficiency and operation times. Operation times is a metric that we did not have the time to explore during this project, but nevertheless a central and critical one in the world of micro drone development.

3 Hardware

The Hardware chapter presents an overview of the electronic components used throughout the iterative development process.

3.1 Embedded board

Raspberry Pi Pico W: from this point on referred to as *"the RPI Pico W"* for short, is a micro controller board utilized within the *Internet of Things* industry. Based on the *RP2040* processor chip, it features a dual-core M0+ processor based on the *ARM* architecture with a maximum clock speed of 133 MHz. The *RPI Pico W* also includes 264 kB of static RAM and 2 MB of flash memory. For external in- and output, the *RPI Pico W* uses 26 *General Purpose Input/Output (GPIO)* pins with select pins having capacity for I^2C, *UART*, *ADC* interfacing and *Pulse With Modulation (PWM)* communication. It also provides fundamental wireless *WiFi* and *Bluetooth* connectivity through the *CYW43439* chip on-board [27].

The *RPI Pico W* covers 3 main objectives; processing incoming control inputs from the pilot, reading sensor orientation data, and executing the flight control software. The objective is to modulate *PWM* signals to the motors, in order to adjust the drone to the desired elevation and direction. This software is rather fundamental, rendering a traditional *Operating System (OS)* redundant. Therefore, the *RPI Pico W* does not incorporate an *OS* for this project, instead executing raw code files through the *micropython* firmware *(more on this in section 4)*.

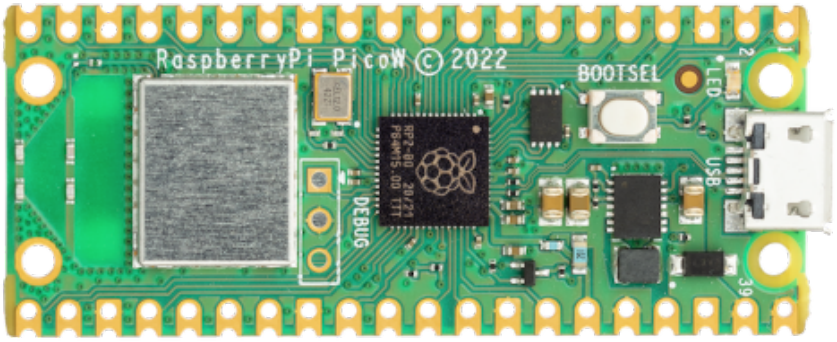
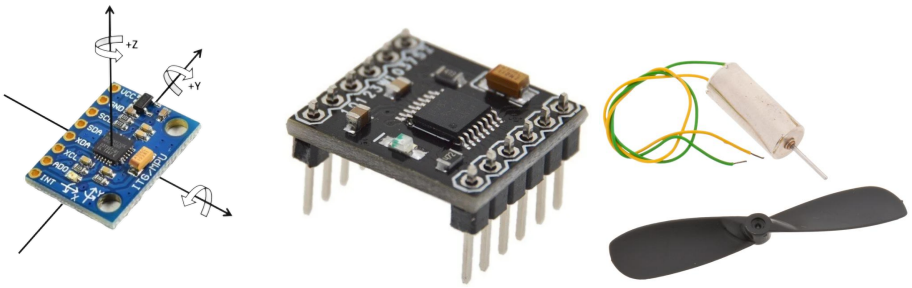


Figure 3.5: The *Raspberry Pi Pico W* board

MPU6050: a gyroscope and 3-dimensional accelerometer combined into a singular module [28], the *Inertial Measurement Unit (IMU)* provides the pitch, yaw and roll angles and speeds required to determine the drone's current orientation and speed. These are utilized for the control of the motors, and the module is communicated to over *I²C* from one of the *RPI Pico W's* able pins.

DRV8833: is an *Electronic Speed Controller (ESC)*, regulating current to up to two *DC* motors. It can be powered by voltages ranging from 2,7V to 10,8V, and has an output current of 1,5A with 2A peaks [29]. By adding two of these *ESC's*, we can control all four motors *(two per DRV8833)* via *PWM* signals from the *RPI Pico W*. The exact balancing and functionality is managed by the navigation software *(section 4)*. To control a motor, the *DRV8833* requires a power wire, a ground wire, and two *PWM* inputs per controlled motor *(controlling the modulation logic)*.



(a) The *MPU6050* module (b) The *DRV8833* module (c) A brushed dc motor

3.2 Output devices

Generic brushed DC coreless motor: is a nameless make of *ø7×16.5mm* brushed motor that was used in the prototyping of this drone. According to the seller they require a supplied voltage of 3.7V, at full load draw 1.3A, and *(without load)* reach 50,000 *RPM* [30]. Four of these motors were connected to the two *DRV8833 ESCs*, in turn modulated by the *RPI Pico W* acting as flight controller *(note figure 1.2)*. In future revisions, motors of another make would be recommended [31], yielding a higher thrust, thereby reducing motor load during flight, and increasing the maximum carrying capacity and flight stability.

3.3 Drone body

The drone prototype is designed from scratch, and therefore *Computer Assisted Design (CAD)* and 3D-printing technology had to be employed, in order to make a custom chassis for it. Using *TinkerCAD*, an online CAD tool, a functional chassis was designed, and later printed. The main parts of a drone body *(see figure 3.7)* are the supports for affixing the motors (1), a case (3) to hold the internal hardware together, a support for holding a battery (4) during flight, and during development an access port to the *RPI Pico W* micro USB port (2). The main considerations are space between the motor supports (1), ensuring that the propellers will have enough clearance, total weight of the body, to ease load on the motors for liftoff and flight (while still maintaining sufficient structural integrity under stress), and overall printability *(a chassis that cannot pass physical production or 3D-printing is of little use)*. For ease of use, and it's wide-spread popularity, matte PLA ø1.75mm plastic [32] was used for most body prototype prints, and it proved the most successful for our usecase.

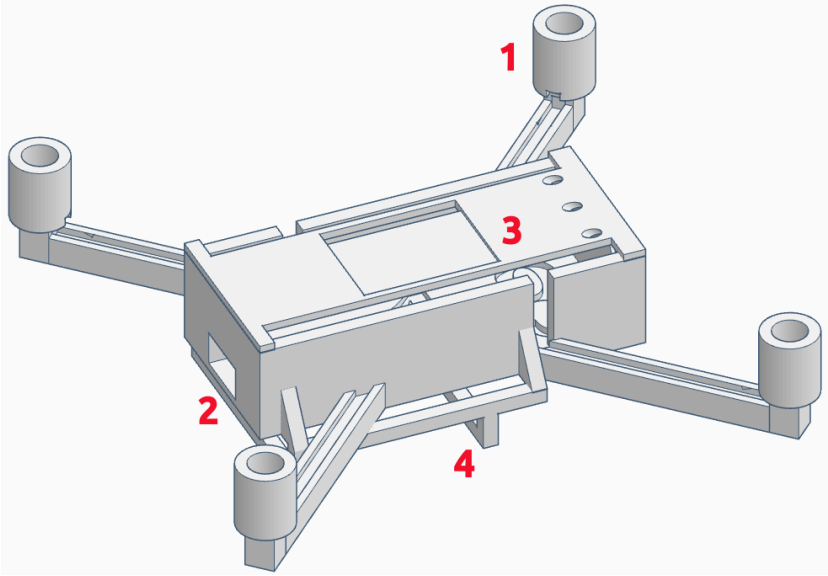


Figure 3.7: The parts of a drone chassis *(prototype 1.3)*

Component	Datasheet	Pinout	QR code
Raspberry Pi Pico W	Raspberrypi.com	picow.pinout.xyz	
MPU6050	TDK/InvenSense	Arduino Forum	
DRV8833	Texas Instruments	Handson Technology	
Brushed motor	electrokit	ElektroPeak	

Table 3.1: Hardware documentation links

3.4 Component table & Circuit diagrams

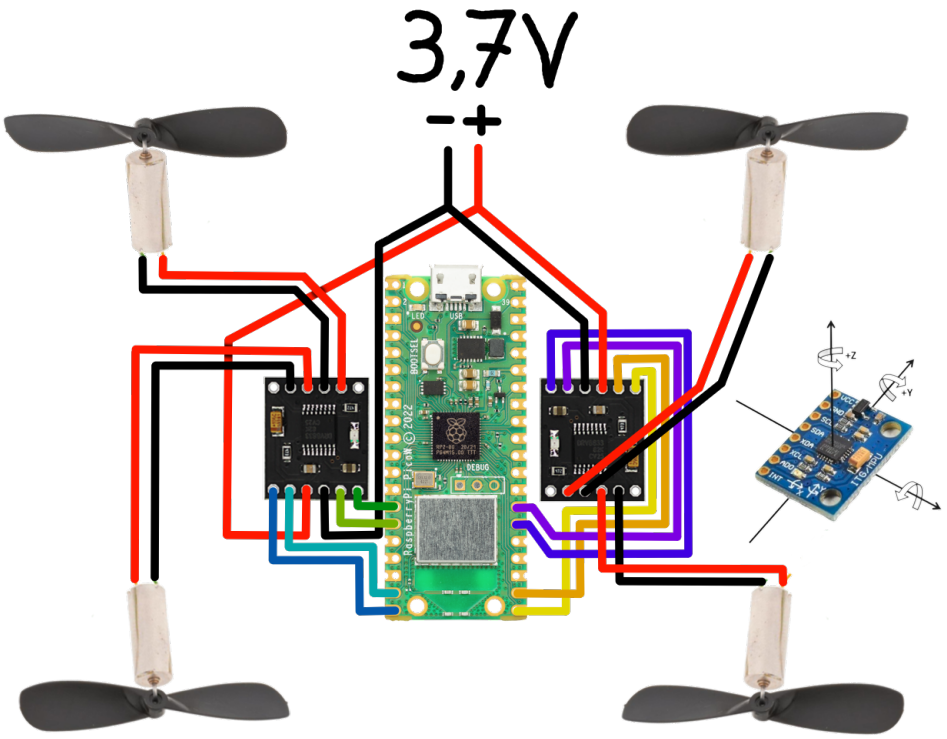


Figure 3.8: Wiring diagram for drone prototype

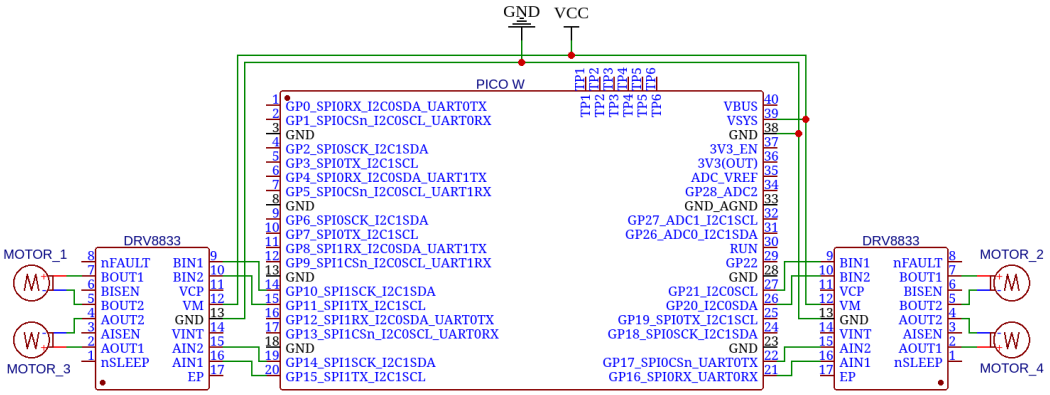


Figure 3.9: Circuit diagram for drone prototype

3.5 Power

The theoretical power consumption of the drone prototype can be calculated as follows:

$$I_{total} = (I_{motor} \times 4) + I_{RPI\ Pico\ W} + (I_{DRV8833} \times 2) + I_{MPU6050} \quad (1)$$

From [30, 27, 28], assuming maximum load *($I_{DRV8833}$ is negligable)*;

$$I_{total} = (1,3A \times 4) + (< 300 \times 10^{-3}) + (0 \times 2) + (3,9 \times 10^{-3} A) \approx 5,2A \quad (2)$$

For the drone, mainly LiPo batteries could be considered due to their weight-to-capacity ratio [26], and the highest-capacity one accessible to us (within the 3.7V requirement from the motors) was a single-cell 3.7V LiPo battery with 750mAh capacity (figure 3.10) and a maximum continuous discharge of 2C *(equivalent to 750 × 2 = 1500 mA)* [33]. It would also yield an estimated battery duration of 8.5 min, after quick calculation *(750mAh / 5200mAh ≃ 8.6min)*.

Figure 3.10: A LiPo battery

For later revisions this battery is clearly not suitable for such a drone, since the maximum current draw is much higher than the battery's maximum discharge.

$$\textbf{5200mA} = 1000 \times 5,2A \gg \textbf{1500mA} \tag{3}$$

Instead, we would recommend the usage of a lighter battery with a drastically higher maximum discharge rate, e.g. this one [34] at 25C (6500mA). Worth considering in that case is buying two and connecting them in series to extend the flight duration, since the discharge rate will empty the already small battery capacity even quicker.

4 Software

This chapter presents the software development of the flight controller. The software development is also divided into two different development phases: Phase 1, consisting of test software for thrust testing, and Phase 2, adding navigational functions and self-correcting stabilization.

4.1 Phase 1 software (thrust testing)

In order to easily understand the structure of the software code, some fundamental theory of aerodynamic principles must first be presented. Aerial vehicles inherit 3 principal axes of rotation, in which 4 movements can be achieved; pitch, roll, yaw and elevation change *(see figure 4.11)*. Pitch is achieved by increasing the force of the rear pair of motors and decreasing the force of the front pair of motors, also causing forward motion. Roll is achieved by increasing the power to the motors on one side of the drone and decreasing to the other side, causing a "side turn" or y-axis movement. Yaw is achieved by increasing power to two diagonally opposite motors while decreasing power to the remaining two, the counter-torque using this angular momentum to rotate the drone [26]. To cause elevation changes you increase the power to all motors, counteracting the gravitational pull. In essence, a drone's navigation is achieved by modulating force vectors caused by the motors' rotations in specific patterns until the desired motion has been achieved.

The phase 1 software constitutes fundamental tests for the flight controller, and it therefore does not include any stabilization technique or user input. For the drone's directional commands it instead utilizes a *super loop* architecture, where the code is executed in a controlled sequence from the onboard micro-python file. The purpose of phase 1 is to evaluate if the drone can achieve lift-off and has capacity to perform the principal axis movements. The controlled sequence secures that these parameters are not affected by external variance or the pilots capacity. The super loop follows the following structure:

1. *Increase throttle simultaneously on all 4 motors until hover*
2. *Decrease throttle on all 4 motors until landing on the ground*

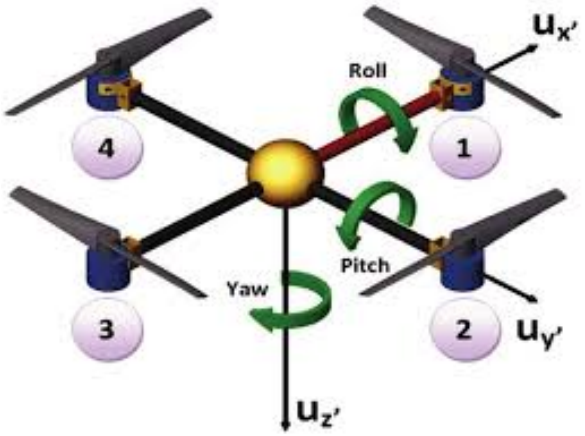


Figure 4.11: A drone's movements

4.2 Phase 2 software (navigation and stabilization)

The phase 2 software utilizes a *closed loop* architecture in order to combine the force vectors from the dc motors in a controlled fashion and avoid destabilization. A closed loop architecture means the software continuously employs the sensor output to adjust its current behavior toward a desired *setpoint*. The control loop structure initiates its iteration by reading a directional command from the pilot. The desired setpoint in this case is a stable angle across all 3 axes (usually meaning only one of the axes has another value than 0). The algorithm employed for this adjustment is *PID*. *"PID is the most common flight control algorithm used in multirotor systems."* [35], and it functions by receiving sensor input data and a setpoint, and minimizing the positional error between the current value and the desired setpoint by adjusting control outputs[26]. PID achieves this by processing the error through a 3 factor algorithm, consisting of the *proportional*, *integral* and *derivative* factors. The *proportional* component reacts to the current error by producing a counter control output, whose strength is relative to the size of the error. The *integral* components summarizes errors over time to eliminate accumulated *steady state* errors *(errors remaining after system "stabilizes", meaning the error value no longer changes)*. The *derivative* component controls the systems rate of change, by reducing control output if the process variable increases by too much. *"The process of setting the optimal gains for P, I and D to get an ideal response from a control system is called tuning"*. These optimal gain values can be obtained trough the *Ziegler Nichols* method or trough trial and error [36].

The current angular value is given by the IMU. The IMU provides two ways to measure orientation: the *gyroscope*, and the *accelerometer*. A gyro-

scope measures *rotational rate*. The current orientation angle is calculated trough integrating the rotational rate by passed time [37]. An accelerometer measures acceleration including *gravitational acceleration*. The current orientation angle is calculated trough measuring the projection of the gravitational acceleration vector across the 3 axes of the sensor [38]. Both of these methods however, posses drawbacks as the accelerometer is sensitive to noise and the gyroscope drifts over time. To solve this, a mathematical method called the complementary filter is applied, which is a method to merge the estimates from a gyroscope and accelerometer, to improve accuracy. A complementary filter functions by *low-pass* filtering the accelerometer measurements *(filtering out spikes)* and *high-pass* filtering the gyroscope measurements *(filtering out drift)* [37].

The execution of the PID algorithm is as follows: the IMU values are continuously read and updated, compared to the setpoint, and then the PID algorithm decreases the difference by decreasing or increasing the individual throttles to the DC motors, by adding the output value from the algorithm, until the setpoint is reached.

The steps in the control loop can be summarized as:

1. *Read pilots input command, and fetch the desired stability setpoint for the movement.*

2. *Read the current orientation value, and filter the value for increased accuracy.*

3. *Compare and minimize setpoint and current value via the PID algoritm. Repeat this until the drone is at target position.*

Phase 2 takes advantage of existing software libraries for reading and filtering the accelerometer and gyroscope, and for implementing the PID algorithm. The primary motivation behind reusing existing code is adhering to the software principle of reinvention of the wheel. When it is quicker to find and reuse an existing solution to a problem than rewriting it from scratch, you will benefit from doing so [39]. Apart from likely also yielding higher quality code, another benefit is a decrease in variance making it simpler to identify the cause of errors. Since time is a constrained resource and the primary research focus for this project is on developing hardware configurations, using libraries was deemed to be both acceptable and necessary. It is worth noticing however, that the whole control loop architecture and pin setup is our own code, though only reaching around 30% of total lines of code throughout the project.

4.3 Programming language

The code is written in micropython and executed using the micropython firmware [40]. Many reasons motivated developing in micro python over alternative high level programming languages, assembly languages, or using real-time operating systems. A primary reason is the nature of the software. It is executed in a sequential, non-changing order for both phase 1 and phase 2. This means that no task scheduling is required as no task holds a higher priority than the others. The drone does not either have a real-time constraint, as the consequence of a delay in milliseconds in control input is insignificant. Due to the drone's low weight and operating force, the drone cannot either in itself be considered a safety critical system. If a very long delay would occur, it would likely generate an error, the damage and severity of which is highly negligible. Since the drone lacks time constraints, the extra performance given by writing in assembly language is not necessary for system functioning, leading to the added development time not being worth it.

4.4 Security threats

The drone possesses no encryption or authentication. The primary reason for this is that it is very unlikely that the system would be a target of hacking or breaching in its current state, especially considering that it does not carry any payload of value. No major security threats that realistically could occur exist. As mentioned, the system is not a safety critical system and any unauthorized control would cause minimal harm.

5 Results and Discussion

Describes the process of initial system design, the project phases, and the iterative drone prototyping process. Accompanying the results are discussions of what improvements and goals are achieved, and notes about encountered errors and problems. It is also here that you will find final material costs, and descriptions of completed tests for the optimization metrics.

5.1 What are phases?

We started work on the report following the project proposal [41] that had been submitted as part of our project selection, and as can be seen below in table 5.2 and figure 5.12, we managed to uphold our scheduled progress up until week 9, save only a few details. But, as further elaborated on later in this report, week 9 and 10 had to be used for additional thrust testing and and awaiting new components.

Due to the nature of *experimental research* it can be hard to draw a line between what is considered designs and redesigns of a prototype, and what is considered iterative improvements. We decided to call these broader concepts *phase 1* and *phase 2*. Different parts of the project (e.g. the software and the chassis design) can be in different phases, and can also go back down to phase 1 temporarily if an iterative step is large enough. We decided to *(for the μUAV prototypes)* draw the line between the *beta series* and *gamma series* (figure 5.12), due to the gamma series beginning the optimization of our first metric -carrying capacity-, by reducing the weight of the drone itself.

Week	Milestone	Description
w5	1: *One propeller works*	Connect to one electrical motor, and set up work repository
w6	2: *Give it a body*	Define list of hardware, design drone case/arms in CAD
w7	3: *Assembled prototype*	Connect to all 4 motors, and mount in/on body
w8	4: *It flies!*	Test carrying capacity, drone case redesigns
w9	5: *It flies where we want it to!*	Test stabilisation/control + knockover tests
w10	6: *We know how long it flies, and where to.*	Test landing + operating time/power draw
w11	7: *Report written*	Draft report, gather additional sources
w12	8: *Construction manual done* 9: *Report polished, ready for hand-in*	Make material list, finish report

Table 5.2: Progress plan and weekly milestones

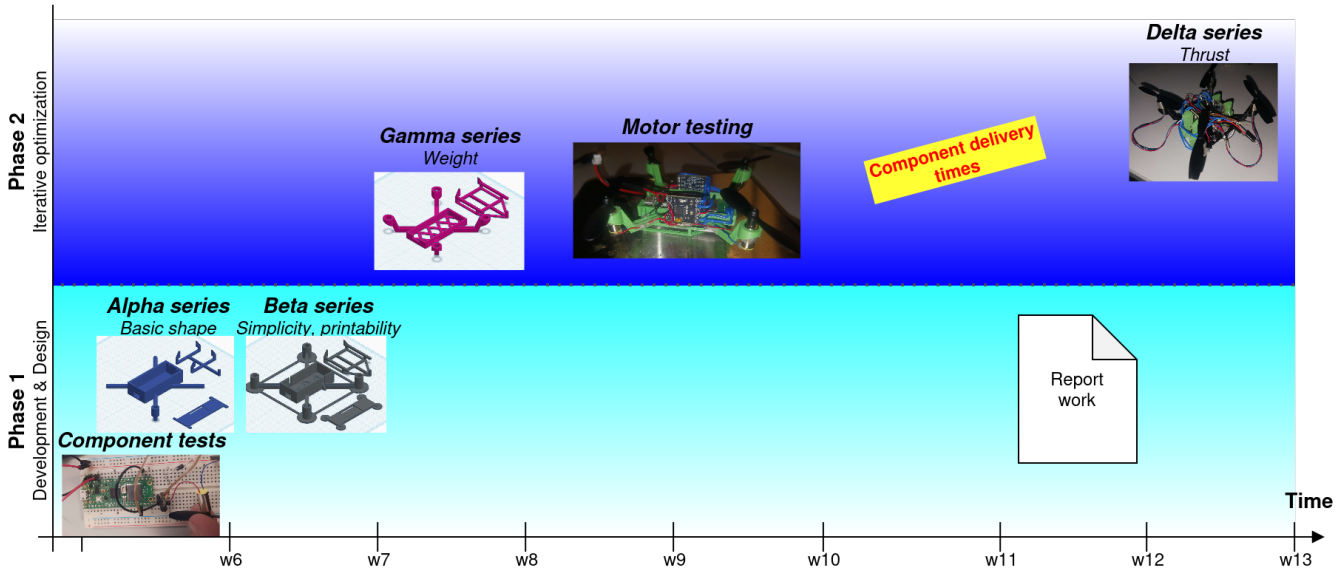


Figure 5.12: Project timeline graph

5.2 Initial system design

As a starting point for our component selection, we selected some available starting components for testing each of them individually, and getting an idea of how to have them interface with each other. We picked up:

- An **RPI Pico W**, due to ease of access, the flexible pinout array, and previous experience using it with *micropython*.
- A gyroscope/accelerometer, the aforementioned **MPU6050**. With many tutorials online, it was quickly set up as per below (figure 5.13), using the *MPU6050 library* [42]. For easy setup, only VCC (power), GND (ground) and SCL and SDA (I²C GPIO pins) were used.

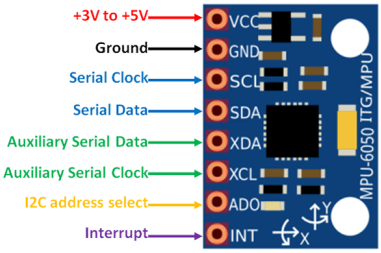


Figure 5.13: The *MPU6050* pinout

- A **generic DC motor**, to test control circuit designs (figure 5.14).

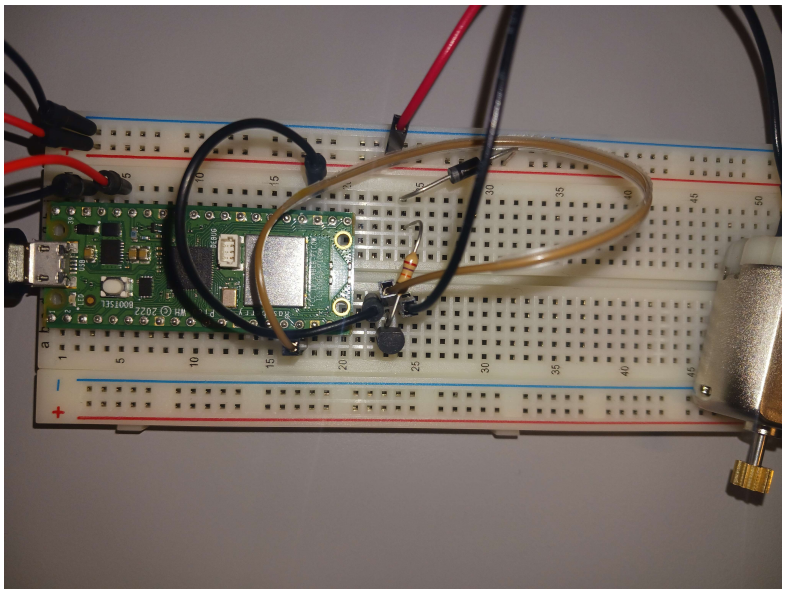


Figure 5.14: Primary motor test

After successfully interfacing with the motor (thereby completing *milestone 1*) and with the IMU, some conclusions were quickly drawn:

1. A drone motor has to have a much higher thrust to weight ratio than this original one.

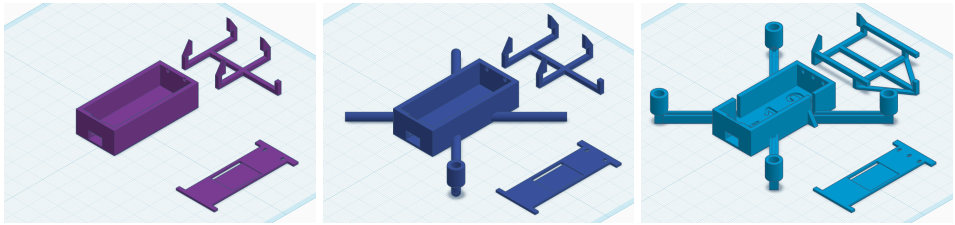
2. A drone motor needs an *electronic speed controller* that can modulate the current to the motor, so that the motors can have different thrusts, a feature required for flight control.

3. We would have to find a battery to power the drone even after successful liftoff, as to not remain connected to the workstation device for power supply.

4. A chassis to hold the components together was needed to progress in component testing.

5.3 Drone prototype: Alpha series

When the conclusions in the initial system plan had been considered, development began with the alpha prototype series. New motors marketed for micro-drone development were ordered [30], the *DRV8833* dual-motor ESC module was found and ordered *(and its library [43])*, a LiPo-battery was acquired (figure 3.10), and a primary chassis design was made in *TinkerCAD*. A μUAV typically has four motors, and so does the first prototype. To balance the four motors, they are affixed to each their own arm, extending out from a central block containing the main electronics (figure 3.7). While developing the alpha prototypes, we went through three versions, v1.1, v1.2 and v1.3, each with their own drawbacks, design decisions and lessons:



(a) Prototype 1.1 (b) Prototype 1.2 (c) Prototype 1.3

Figure 5.15: The *Alpha* chassis prototype series

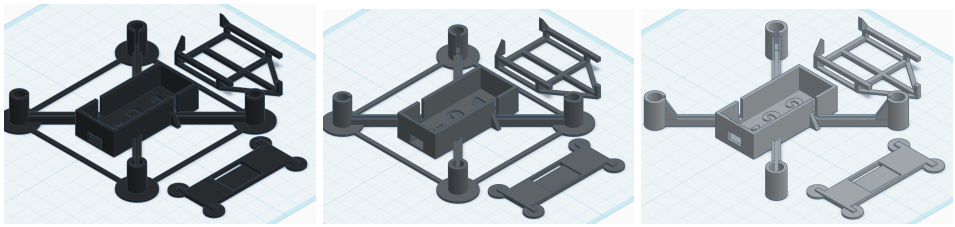
With v1.1 (a), we chose to have the battery held in an external cage, to ease battery installation and replacement, and to have a fully removable lid

to the central box. For v1.2 (b), we realized that the motors were going to need arms and sockets to be able to be well installed. Then with v1.3, we made printability easier by making the arms square in place of the previous rounded shape, adding the final motor sockets, and some grooves and slits for wiring from the motors. The back battery cage fixation points were also moved onto the chassis, and support beams added between the bars, to ease its printability. Due to persistent issues with the 3d-printer *(BambuLabs X1 Carbon, default PLA settings)*, no alpha series drone prototype was ever physically assembled.

5.4 Drone prototype: Beta series

To try and avoid the printing issues (mainly loosing *build-plate adhesion*), more supporting bars were added between the chassis arms, and sharp corners had supporting plates added around them. We also tried switching to another 3d-printer *(Prusa Mini, almost default PLA settings)*, an approach that proved successful. Note that the IMU would not be installed until a prototype proved liftoff.

```
# Successful Prusa Mini settings used:
Smooth PEI plate
1,75 mm matte PLA (green)
0.4mm nozzle ø
0.2mm layer height
```



(a) Prototype 2.1 (b) Prototype 2.2 (c) Prototype 2.3

Figure 5.16: The *Beta* chassis prototype series

To ensure printability in v2.1 (a), we added support beams between the motor arms, and circular disks at each sharp corner. We did get a successful lid print. With v2.2 (b) we switched to the Prusa printer, and slightly adjusted the size of the lid socket for the IMU, we also tried printing with *glossy PLA* in place of normal *matte PLA*. The glossy PLA proved much more brittle than matte PLA, so we returned to using matte PLA, getting our first fully

successful print. Then with v2.3 the supporting beams from the main chassis were removed, since the printability issues proved to stem from the printer itself, and not the 3d model.

A beta series prototype was then assembled, with components being soldered with headers intended for reusability during development (figure 5.17, finishing *milestone 3*). The circuit was configured in accordance with the diagrams in figures 3.8 and 3.9. Upon running the test program (Appendix B), successful driver instructions to motors were achieved and executed, and the motors were powered by the battery. One weaker propeller could be noticed (blamed on lack of voltage in the system), and no liftoff was achieved. We therefore decided to start reducing the system weight to try and ease liftoff.

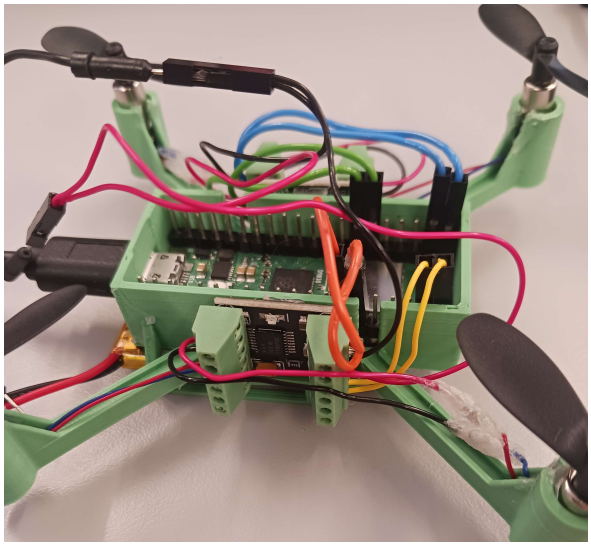


Figure 5.17: The assembled *Beta* prototype

5.5 Metric optimization

To properly respond to our optimization parameters *(cost effectiveness, design simplicity, carrying capacity, and flight stability)*, defined tests for each of them were made.

1. **Cost Effectiveness**: An overall evaluation considering the total prototype component cost, what performance it delivers, and what methods are used in its construction.

2. **Design simplicity**: An overall evaluation considering the number of components involved in the prototype, the ease of installation or replacement of parts, and accessibility to the general public.

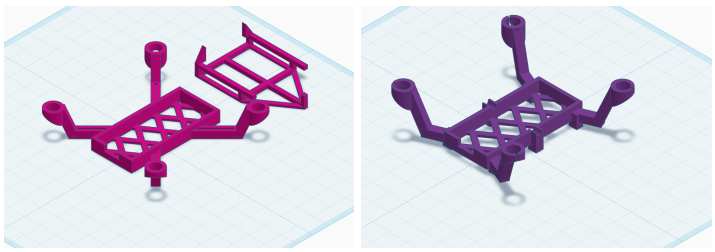
3. **Carrying Capacity**: A test where the prototype is placed on a scale, measuring maximum downwards thrust, maximum total weight with successful liftoff, and maximum total weight for a stable hover.

4. **Flight Stability**: A test where the prototype attempts to lift off, hover in place, and then land. It is then repeated when adding navigation instructions, and repeated again with external disturbances.

The metrics subject to analysis, when each prototype is compared to each other, and to specifications from a drone option already available on the market, contribute to classifying the analysis section of this project as an inductive analysis; *"...inductive content analysis involves collecting and analyzing data without preconceived categories or theories. This flexibility allows the data to guide the researcher's analysis in order to identify emerging patterns, themes, and concepts."* [44]. Resulting in a comparative analysis of development parameters and prototyping results, this project contributes *explanatory knowledge* to the field through insights gained by reading this report.

5.6 Drone prototype: Gamma series

When we were content with the composition of the chassis (completing *milestone 2*), we moved onto reducing the weight. Based around a lattice structure for maintaining the chassis structural strength while removing much surface area, its weight could be greatly cut down. Since we still had to uphold our metric of design simplicity, we opted to not modify or replace our component list as to keep utilizing ones more easily accessible by later on people replicating the drone. That lead to three areas remaining to cut down weight at; the chassis itself, the component headers, and the wires. At this stage we hade to sacrifice an aspect of *design simplicity* to be able to continue optimizing the other three metrics, and work towards liftoff, and we chose to assume that people replicating this project have the tools and knowledge required to solder the components tightly in place.



(a) Prototype 3.1 (b) Prototype 3.2

Figure 5.18: The *Gamma* chassis prototype series

With the v3.1 (a) prototype the chassis, the box walls were lowered, the box bottom made into a lattice, the motor arms slightly shortened, and since the IMU wasn't installed the lid was entirely removed. Then, wires were installed in a much tighter fashion than for the beta prototype (figure 5.19), still adhering to the same circuit diagram as in figure 3.9.

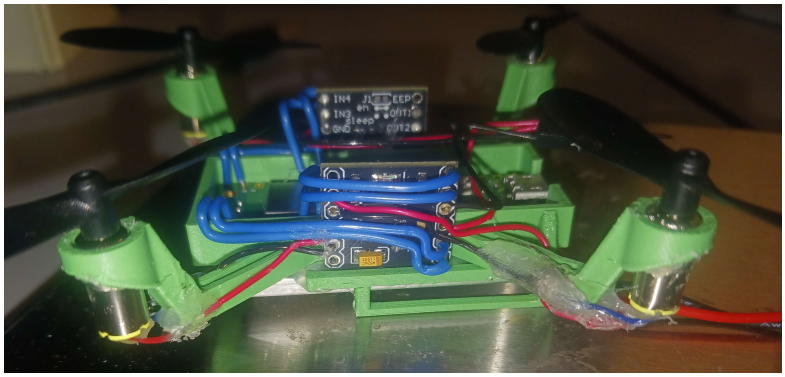


Figure 5.19: The assembled *Gamma* prototype

Around this point, during testing (for *milestone 4*), we assumed that the motors we had bought were too weak to provide the required thrust. After inquiry with the reseller, this theory was confirmed;

> *"Hi, we can get you a refund, regrettably our supplier has changed the motors against new worse ones, without informing us. They have been better before."*
> *–Electrokit customer support [45]*

This setback led to us not having the time to complete *milestones 5* and *6*, due to long delivery times upon ordering new ones [31]. We also suspected that the system was receiving too little current from the battery, so we switched to a lab *PSU* [46] for further testing.

Another version, v3.2 (b), was later made. Since there were few major changes to the actual chassis, we considered it still part of the gamma series, even though this gamma v3.2 chassis was made for the *Delta* version of the drone prototype.

5.7 Drone prototype: Delta series

When the new propellers arrived, we put together a quick prototype in a gamma v3.1 chassis, but using one ESC per motor. Thereby we hoped bypass the limit of 1,5A per motor (if two motors are connected simultaneously) [29], yet another theory that proved correct.

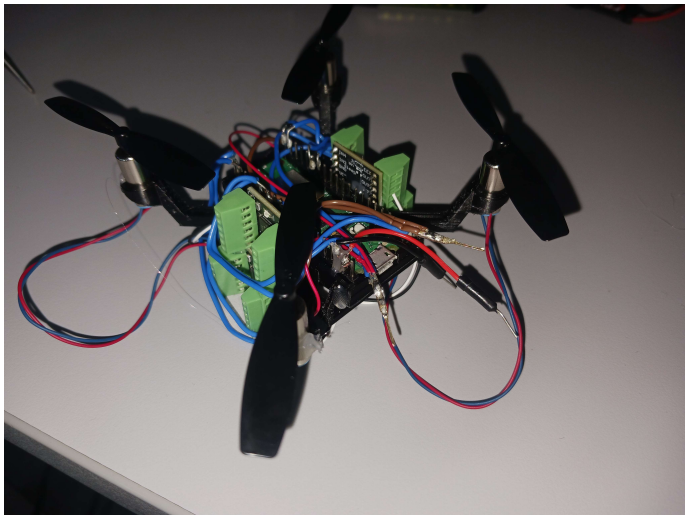


Figure 5.20: The assembled *Delta v4.1* prototype

While this prototype is not as optimized for total weight as its predecessor, nor as well assembled, it did have one great advantage over it; liftoff from the ground was successful (video: [47], test sequence in Appendices A and C). The total downwards thrust was measured to 30g and the upwards thrust must have surpassed 46g, since that was the weight of the prototype, and it lifted off of the scale. To try and further optimize the carrying capacity (by increasing excess thrust), the gamma v3.2 chassis was designed with a wider angle between the drone arms to fit the ESC boards side-by side. No functional prototype of this Delta v4.2 prototype was ever assembled, although an attempt was made (figure 5.21). The IMU was installed, and tight wiring soldered on, for an estimated $\sim$35g total weight, but adapted for PSU current (no battery). However, during testing, there was irregular responses from the motors and unexpected behaviors overall, leading us to guess there was a short-circuit somewhere in the prototype. We did not have the time to investigate further, or make a new prototype.

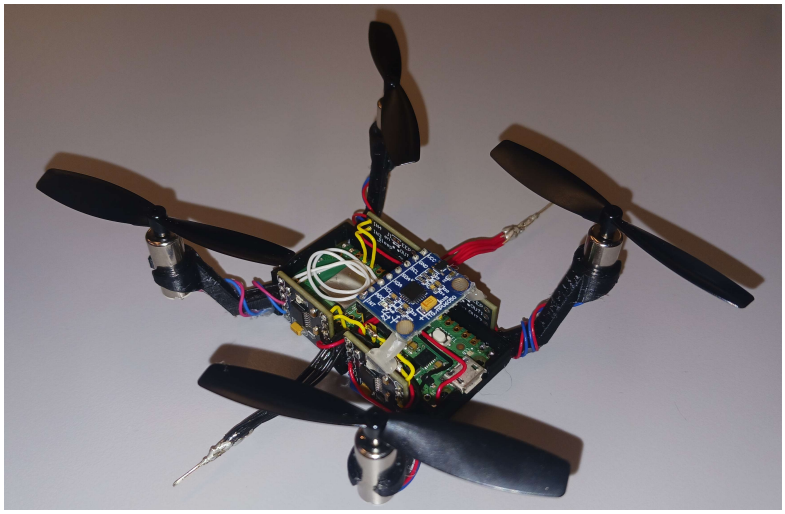


Figure 5.21: The assembled, non-functional, *Delta v4.2* prototype

5.8 Evaluation of metrics

Based on measurement gathered during the iterative prototyping, the below table of comparison (table 5.3) was assembled for final analysis and evaluation.

Metric	**Crazyflie 2.1+**	**Alpha**	**Beta**	**Gamma**	**Delta**
Number of parts (pcs)	12	12	11	10	12
Total cost (SEK)	2266,93	467,66	402,18	400,02	335,50
Downwards thrust (g)	unknown	n/a	6	2	30
Carrying capacity (g)	15	n/a	n/a	n/a	0+
Total weight (g)	29	∼49,0+	53,0	35,5	46
Flight duration (min)	7	n/a	n/a	n/a	0,03

Table 5.3: Optimization parameter results

Flight stability is perhaps the easiest metric to evaluate, but at the same time the most difficult to accept; Neither prototypes alpha (never assembled), beta or gamma managed to lift off of the table, and then, right as the allocated project time ran out, prototype delta succeeded in a liftoff. Barely anything worth comparing to the already market-proven *Crazyflie 2.1+*, but nevertheless an inspiring result if further research is done on this this track. Had we only had more time to develop a delta v4.3, or even just debug delta v4.2, implementation of stabilisation software could have been started. As explained in the software section, pseudocode for stabilisation and navigation had been

written as an elaboration and practical implementation of the test program (Appendix B).

Carrying capacity has a similar result as flight stability, due to the same thrust issues. Downwards thrust was measured for each prototype, and began producing clear and usable amounts at first for prototype delta v4.1 when each motor had its own ESC. What can be noted is the correlation between current to the motors, and resulting increase in downwards thrust, which could have been expected. The big jump in downwards thrust between prototypes gamma and delta can in part be explained by the new motors having slightly bigger propellers, as can be seen in figure 5.19 and 5.20. For future reference it is also worth noting that the weight of prototype delta v4.1 could reasonably have been cut down by possibly as much as 10g (based off of v4.2 and v3.1), possibly matching the 15g carrying capacity of the *Crazyflie 2.1+*.

Design simplicity remains as one of the strongest suits of the prototypes (*Number of parts* in table not including propellers or wires). Having about the same number of total components as the *Crazyflie 2.1+*, even though it has a custom PCB flight controller, and using more easily accessible components makes our drone about as repairable and replicable as the *Crazyflie*, without their extensive sale of spare parts. Soldering a full prototype has been done in about 4h while working systematically, and only requires wire and soldering equipment. The chassis itself is designed in CAD, and can be rather easily be printed on a bare-bones FDM printer such as the Prusa Mini with normal PLA. A glue gun sufficed for assembling non-conductive components.

And lastly; **Cost effectiveness**. We almost managed to replicate the core functionality of a μUAV for 15% the cost of the market-released *Crazyflie 2.1+* (calculated comparing component cost [Appendix E] vs web store sale price). This project shows that by ordering in bulk, and streamlining the production process even more, there is a chance to undercut the price of commercially available drones. To do so without losing too much of the drone's utility, and while increasing ease of repair and component procurement shows that the drone development field still has a lot of improvements to do. Hopefully this report can spread access to cheap drones to scientists across the world, and show drone developers that there are still improvements to be done.

6 Conclusion

This section explains how work was distributed within the group, shares the challenges encountered during the project, and has some tips for future work.

During the first half of the project, work was slightly divided among the members of our group. The hardware and soldering aspects were mostly handled by Noa, whilst the software and navigation program was handled by Samuel. The background research and report writing was done in tandem with both members working simultaneously, leading to an equal time spent of slightly over 100 hours each (Appendix D).

A number of challenges arose during the project duration, their solutions being the scientific contributions this report provides. One major factor that negatively affected the research progress where components. Due to unknown factors, key components including appropriate coreless dc motors where very **difficult to source**, as they were not available for purchase from any major Swedish retailer or distributor. This affected the project by forcing the development to use components from third-party markets including Amazon and AliExpress. Consequently this lead to **low quality components** with **high delivery times** and/or significant costs compared to offered value (several components also had to be refunded). Our component collection also suffered from the **low development budget**, leading to lack of backup parts and even lower quality sourcing.

The **small scale** required for the manual soldering caused increased risks of **assembly errors**, that cost time to resolve. As each new iteration of prototypes presented a new component configuration, new chassis-es **constantly** had to be designed and the **prototypes reassembled** and soldered.

Possibly the most significant issue of this project, was **insufficient power supply** to the motors. During the latter end of the testing phase, it was noticed that the maximum current from the battery had to be concernedly high to provide the necessary current demands of running the 4 motors at full thrust. This successionally led to **stressing the ESC maximum current** limit. In future revisions of this project, continued testing of current distribution has to be performed, in order to reach the thrust levels required for stable flight.

Several issues arose during the 3d-printing process of the chassis, with the configuration of the Lab 3d-printer, the *BambuLabs X1 Carbon*. The printer was unable to reach the required levels of **build plate adhesion** to successfully dispense the initial layers of filament, leading to *spaghetti printout*. After several retries and reconfigurations, exploring nozzle changes, material variations and model printability improvements, it was deemed that the

printer was non-functional. The final solution was to replace the *BambuLabs X1 Carbon*, and setup another, more manual printer. It remains unknown if the source of the failures were due to a damaged print plate, faulty temperatures, or some other issues with the specific *BambuLabs X1 Carbon* lab device, but using the simpler *Prusa Mini* proved successful.

For continued work with this project, we have some recommendations:

- Look into replacing the *DRV8833* electronic speed controllers, as they barely support the currents required for full motor thrust, even with one motor per ESC. E.g. using the DRV8871 [48] with a higher maximum current limit.
- Find a better battery, providing a higher maximum current draw value, perhaps this one [34] with 25C in place of the current 2C.
- When powering the drone by battery, develop an internal battery system for calculating remaining battery percentage, and a charging module.
- Optimize the flight controller processing unit by comparing different MCU options. To have a whole RPI Pico W board in the chassis center while only utilizing some of its pins is suboptimal. It is worth looking into designing a custom PCB to reduce weight.
- Compare efficiency of different sensor fusion algorithms, such as the *Kalman filter*, in place of the current complementary filter.
- Make a live controller system to provide navigational instructions from the pilot to the drone. For example a mobile app with a user-friendly GUI.

To conclude, when optimizing towards cost effectiveness, design simplicity, carrying capacity and stability factors a number of challenges are encountered. By balancing them based on the desired area of application, sacrifices in one parameter can improve upon another. By reducing the design simplicity and centralizing the flight control functionality (e.g. in a custom PCB), stability and cost effectiveness can be increased, and by reducing the cost effectiveness (e.g. increasing the budget), carrying capacity, flight stability and design simplicity can be increased due to more monetary margins for better components.

The micro drone development field is affected by the interaction of many sciences; aerodynamics, electronic components, prototype configuration, flight

controllers and software like sensor fusion algorithms and adjustment algorithms. With this project, we explore some of these interactions, reporting our results to contribute towards bridging this today (more than ever) relevant knowledge gap.

Since the development of an optimal drone for any purpose is an impossible task by nature, it requires constant research to strive as close to that goal as possible. But even though unobtainable, the discovery of new technologies promise the continued chance for progress.

> *"On difficult ground, I would keep pushing on along the road."*[49]
> *- Sun Tzu, The Art of War*

References

[1] M. Hassanalian and A. Abdelkefi, "Classifications, applications, and design challenges of drones: A review," *Progress in Aerospace Sciences*, vol. 91, pp. 99–131, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0376042116301348

[2] M. Arjomandi, S. Agostino, M. Mammone, M. Nelson, and T. Zhou, "Classification of unmanned aerial vehicles," *Report for Mechanical Engineering class, University of Adelaide, Adelaide, Australia*, pp. 1–48, 2006. [Online]. Available: https://www.academia.edu/2055673/Classification_of_Unmanned_Aerial_Vehicles

[3] N. Akram, M. Khoshrangbaf, M. Challenger, and O. Dagdeviren, "Energy consumption modeling and flight time analysis of micro drones," *IEEE Access*, vol. 13, pp. 109 854–109 866, 2025.

[4] Fact.MR, "Micro drone market outlook (2025 to 2035)," *FACT7897MR*, 2025. [Online]. Available: https://www.factmr.com/report/micro-drone-market

[5] M. Emimi, M. Khaleel, and A. Alkrash, "The current opportunities and challenges in drone technology," *International Journal of Electrical Engineering and Sustainability*, vol. 1, no. 3, p. 74–89, Jul. 2023. [Online]. Available: https://ijees.org/index.php/ijees/article/view/47

[6] S. Heath, *Embedded Systems Design*, 2nd ed., ser. EDN Series for Design Engineers. Newnes, 2003.

[7] UMILES Group, "What is a drone and what is it used for? characteristics and functions," https://umilesgroup.com/en/what-is-a-drone-and-what-is-it-for-characteristics-and-functions/, Aug. 2023, accessed: 2026-03-06.

[8] M. Karahan, "Development stages of quadrotors from past to present: A review," *Drones*, vol. 9, p. 840, 12 2025.

[9] B. Belton and L. Baldiga, "Agricultural drones are taking off globally, saving farmers time and money," *The Conversation*, Oct. 2025, accessed: 2026-03-08. [Online]. Available: https://theconversation.com/agricultural-drones-are-taking-off-globally-saving-farmers-time-and-money-265154

[10] "Agricultural spraying drones: A comprehensive review," *Smart Agricultural Technology*, 2025, accessed: 2026-03-08. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2772375525007506

[11] S. Fei, G. Shao, J. P. Hupy, and J. Gallion, "Tree inventory with aerial remote sensing," Purdue University Institute for Digital Forestry, accessed: 2026-03-08. [Online]. Available: https://ag.purdue.edu/digital-forestry/projects/tree-inventory-with-aerial-remote-sensing.html

[12] UAV Coach, "Search and rescue drones: A guide to how SAR teams use drones in their work," https://uavcoach.com/search-and-rescue-drones/, Jan. 2026, accessed: 2026-03-08.

[13] FMV Kommunikation, "Ett vakande öga," *Materiel*, no. 4, 2025, describes incorporation of UAV 06 Skatan (Parrot ANAFI USA Gov) into Swedish Armed Forces service.

[14] D. Kunertova, "The war in ukraine shows the game-changing effect of drones depends on the game," *Bulletin of the Atomic Scientists*, vol. 79, no. 2, 2023. [Online]. Available: https://www.tandfonline.com/doi/full/10.1080/00963402.2023.2178180

[15] A. Usama, M. Dora, M. Elhadidy, H. Khater, and O. Alkelany, "First person view drone - fpv," *The International Undergraduate Research Conference*, vol. 5, no. 5, pp. 437–440, 2021. [Online]. Available: https://iugrc.journals.ekb.eg/article_246400.html

[16] J.-H. Lee, S. Jung, J. Jang, Y. Han, M. Kang, J. Seong, S. Seo, E. Park, H. Kwak, and M.-W. Han, "Design challenges to expand the functionality of drones: Deformable rotorcraft and nature-inspired flapping drones," *Advanced Intelligent Systems*, Nov. 2025, early View. [Online]. Available: https://advanced.onlinelibrary.wiley.com/doi/full/10.1002/aisy.202500790

[17] S. Saharan, M. Khiria, T. R. Chandrakar, M. Sharma, and J. Binwal, "Role of drones in climate change mitigation and adaptation," in *Artificial General Intelligence-Based Drones for Climate Change*. IGI Global Scientific Publishing, 2025, pp. 41–70.

[18] P. V. Singh, "The small-world effect: The influence of macro-level properties of developer collaboration networks on open-source project success," *ACM Trans. Softw. Eng. Methodol.*, vol. 20, no. 2, Sep. 2010. [Online]. Available: https://doi.org/10.1145/1824760.1824763

[19] U. R. Mogili and B. B. V. L. Deepak, "Review on application of drone systems in precision agriculture," *Procedia Computer Science*, vol. 133, pp. 502–509, 2018, international Conference on Robotics and Smart Manufacturing (RoSMa2018). [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050918310081

[20] B. Aydin and S. Singha, "A regression analysis study examining the price of drones," 03 2020.

[21] Bitcraze AB, "Crazyflie 2.0," https://www.bitcraze.io/products/old-products/crazyflie-2-0/, 2026, discontinued. Accessed: 2026-03-24.

[22] ——, "Crazyflie 2.1," https://www.bitcraze.io/products/old-products/crazyflie-2-1/, 2026, discontinued. Accessed: 2026-03-24.

[23] ——, "Crazyflie 2.1+," https://www.bitcraze.io/products/crazyflie-2-1-plus/, 2026, accessed: 2026-03-24.

[24] ——, *Crazyflie 2.1+ Datasheet*, 6 2024, rev 1, Hardware revision B1. Accessed: 2026-03-24. [Online]. Available: https://www.bitcraze.io/documentation/hardware/crazyflie_2_1_plus/crazyflie_2_1_plus-datasheet.pdf

[25] ——, "Crazyflie 2.1+ – bitcraze store," https://store.bitcraze.io/products/crazyflie-2-1-plus, 2026, sKU: 114993491. Price: $240.00 USD. Country of origin: Vietnam. Accessed: 2026-03-24.

[26] A. Muhamad, S. D. Panjaitan, and R. R. Yacoub, "Design and development of flight controller for quadcopter drone control," *Telecommunications, Computers, and Electricals Engineering Journal (TELECTRICAL)*, vol. 1, no. 3, pp. 279–291, feb 2024. [Online]. Available: https://jurnal.untan.ac.id/index.php/TELECTRICAL/article/view/73681

[27] Raspberry Pi Ltd, "Pico microcontroller boards," https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html, 2026, accessed: 2026-03-17.

[28] InvenSense Inc., "Mpu-6000 and mpu-6050 product specification (rev. 3.4)," https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf, 2013, document No. PS-MPU-6000A-00. Accessed: 2026-03-17.

[29] Texas Instruments, “Drv8833 dual h-bridge motor driver datasheet (rev. e),” https://www.ti.com/lit/ds/symlink/drv8833.pdf, 2015, accessed: 2026-03-17.

[30] Electrokit, “Dc-motor 3.7v 50000rpm med fläktblad (art. nr. 41016960),” https://www.electrokit.com/dc-motor-3.7v-50000rpm-med-flaktblad, 2024, product discontinued. Accessed: 2026-03-17.

[31] Motor Kindom Store, “716 coreless motor 7mm×16mm 2cw+2ccw dc 3.7v 55000rpm with propellers (4pcs),” https://www.aliexpress.com/item/1005005201519287.html, 2024, aliExpress. Accessed: 2026-03-17.

[32] Clas Ohlson, “Pla 3d-filament universal, 1.75mm, 1kg (art. nr. 38-7986-1),” https://www.clasohlson.com/se/PLA-3D-filament-Universal,-1,75-mm,-1-kg/p/38-7986-1, 2024, accessed: 2026-03-17.

[33] Electrokit, “Batteri lipo 3.7v 750mah – jst-ph (art. nr. 41016063),” https://www.electrokit.com/batteri-lipo-3.7v-750mah-jst-ph, 2024, accessed: 2026-03-17.

[34] ——, “Battery lipo 3.7v 260mah high-current (art. nr. 41016066),” https://www.electrokit.com/batteri-lipo-3.7v-260mah-hog-stromstyrka, 2024, 25C max discharge (6500 mA). Accessed: 2026-03-17.

[35] A. Noordin, M. A. Mohd Basri, and Z. Mohamed, “Real-time implementation of an adaptive pid controller for the quadrotor mav embedded flight control system,” *Aerospace*, vol. 10, no. 1, 2023. [Online]. Available: https://www.mdpi.com/2226-4310/10/1/59

[36] NI National Instruments corp., “The pid controller theory explained,” 7 Match 2025. [Online]. Available: https://www.ni.com/en/shop/labview/pid-theory-explained.html

[37] V. H. Adams, “Complementary filters,” https://vanhunteradams.com/Pico/ReactionWheel/Complementary_Filters.html, 2021, expository webpage. Cornell University. Accessed: 2026-03-24.

[38] C. J. Fisher, “Using an accelerometer for inclination sensing,” Analog Devices, Inc., One Technology Way, Norwood, MA 02062-9106, U.S.A., Application Note AN-1057, 2010, rev. 0. Accessed: 2026-03-24. [Online]. Available: https://www.analog.com/en/resources/app-notes/an-1057.html

[39] C. W. Krueger, “Software reuse,” *ACM Comput. Surv.*, vol. 24, no. 2, p. 131–183, Jun. 1992. [Online]. Available: https://doi.org/10.1145/130844.130856

[40] George Robotics Limited, “Micropython for raspberry pi pico w,” https://micropython.org/download/RPI_PICO_W/, accessed: 2026-03-17.

[41] N. Cluer and S. Andersson, “Project proposal,” Linnaeus University, Project Proposal, 2026, version 1. Available at https://gitlab.lnu.se/nc222iw/droneproject/-/blob/main/doc/Project%20proposal%20nc222iw,%20sa225hh-v1.pdf.

[42] T. Hanewich, “MPU6050.py: A lightweight MicroPython implementation for interfacing with an MPU-6050 via I2C,” https://github.com/TimHanewich/MicroPython-Collection/blob/master/MPU6050/MPU6050.py, 2023, accessed: 2026-03-24.

[43] AHSPC, “DRV8833_micropython,” https://github.com/AHSPC/DRV8833_micropython, 2024, accessed: 2026-03-24.

[44] Delve, L. Ho, and A. Limpaecher, “Inductive content analysis & deductive content analysis in qualitative research,” https://delvetool.com/blog/inductive-content-analysis-deductive-content-analysis, Mar. 2023, accessed: 2026-03-25.

[45] M. Wainwright, “Personal correspondence regarding DC motor lyftkraft (DRV8833),” 2 2026, email from Max Wainwright, Kundtjänst, Electrokit Sweden AB (`desk.swe@electrokit.se`) to Noa Cluer. February 18, 2026. Private correspondence.

[46] Good Will Instrument Co., Ltd., *GPC-3030 / GPC-3030D Dual/Triple Output Linear DC Power Supply User Manual*, GW Instek, No.7 Jhongsing Rd., Tucheng City, Taipei County 236, Taiwan, analog dual-channel variant (GPC-3030). Discontinued. Accessed via Scribd: https://www.scribd.com/doc/261314561/Gw-Instek-GPC-3030D-User-Manual. Accessed: 2026-03-24.

[47] E. Sundström, “Prototype liftoff test — first successful lift,” Video recording, https://drive.google.com/file/d/1A3VEWKu5BGYKLudTwh_lf8UBIrs_IcV1/view?usp=sharing, 2026, accessed: 2026-03-24.

[48] Texas Instruments, “DRV8871 3.6-a brushed DC motor driver with internal current sense (PWM control),” 2015, datasheet Rev. B (orig. August 2015, revised July 2016). Document SLVSCY9B. Accessed:

2026-03-24. [Online]. Available: https://www.ti.com/product/DRV8871

[49] Sun Tzu, *The Art of War & Other Classics of Eastern Philosophy*, P. Norton, Ed. Canterbury Classics, 2016, ch. 11 “The Nine Situations”, statement 49: “On difficult ground, I would keep pushing on along the road.”. Translation originally published 1910.

A main.py (updated version of drv8833_test.py)

```
from machine import Pin, PWM
from drv8833 import DRV8833 # ../lib/drv8833.py
from time import sleep

Pin("LED").on()
sleep(1)
Pin("LED").off()

# NOTE: feel free to play with the frequency value!
# The frequency (how many PWM cycles per second
# sent to the DRV pins) changes the behavior of
# the connected motors, especially at low throttle
# values. (based on very brief testing) We recommend
# values no lower than 1,000 and no higher than
# 200,000. 40,000 is a good default.
frequency = 40_000

# Make sure to set the correct pins!
r_a_in1 = PWM(Pin(10, Pin.OUT))
r_a_in2 = PWM(Pin(11, Pin.OUT))
r_a_in1.freq(frequency)
r_a_in2.freq(frequency)

r_b_in1 = PWM(Pin(14, Pin.OUT))
r_b_in2 = PWM(Pin(15, Pin.OUT))
r_b_in1.freq(frequency)
r_b_in2.freq(frequency)

l_a_in1 = PWM(Pin(21, Pin.OUT))
l_a_in2 = PWM(Pin(20, Pin.OUT))
l_a_in1.freq(frequency)
l_a_in2.freq(frequency)

l_b_in1 = PWM(Pin(17, Pin.OUT))
l_b_in2 = PWM(Pin(16, Pin.OUT))
l_b_in1.freq(frequency)
l_b_in2.freq(frequency)

drv_left = DRV8833(l_a_in1, l_a_in2, l_b_in1, l_b_in2)
```

```
drv_right = DRV8833(r_a_in1, r_a_in2, r_b_in1, r_b_in2)

MAX_THROTTLE = 0.9#0.99  # maximum throttle
sleep_hold = 8.0
STEP_SIZE = 0.05
STEPS = int(MAX_THROTTLE/STEP_SIZE)

try:
    while True:
        for i in range(STEPS + 1):
            throttle = i / STEPS * MAX_THROTTLE
            drv_left.throttle_a(throttle)
            drv_left.throttle_b(throttle)
            drv_right.throttle_a(throttle)
            drv_right.throttle_b(throttle)

            # small delay to avoid busy looping
            sleep(0.05)
            print("Speed:", throttle)

        sleep(sleep_hold)

        for i in range(STEPS, -1, -1):
            throttle = i / STEPS * MAX_THROTTLE
            drv_left.throttle_a(throttle)
            drv_left.throttle_b(throttle)
            drv_right.throttle_a(throttle)
            drv_right.throttle_b(throttle)
            sleep(0.05)
            print("Speed:", throttle)

        sleep(sleep_hold)

except KeyboardInterrupt:
    drv_right.stop_a()
    drv_right.stop_b()
    drv_left.stop_a()
    drv_left.stop_b()

    drv_right.deinit()
```

```
drv_left.deinit()
```

B drv8833_test.py (beta prototype series version)

```
from machine import Pin, PWM
from drv8833 import DRV8833 # ../lib/drv8833.py
from time import sleep

Pin("LED").on()
sleep(1)
Pin("LED").off()

# NOTE: feel free to play with the frequency value!
# The frequency (how many PWM cycles per second
# sent to the DRV pins) changes the behavior of
# the connected motors, especially at low throttle
# values. (based on very brief testing) We recommend
# values no lower than 1,000 and no higher than
# 200,000. 40,000 is a good default.
frequency = 40_000

# Make sure to set the correct pins!
r_a_in1 = PWM(Pin(15, Pin.OUT))
r_a_in2 = PWM(Pin(14, Pin.OUT))
r_a_in1.freq(frequency)
r_a_in2.freq(frequency)

r_b_in1 = PWM(Pin(11, Pin.OUT))
r_b_in2 = PWM(Pin(10, Pin.OUT))
r_b_in1.freq(frequency)
r_b_in2.freq(frequency)

l_a_in1 = PWM(Pin(17, Pin.OUT))
l_a_in2 = PWM(Pin(16, Pin.OUT))
l_a_in1.freq(frequency)
l_a_in2.freq(frequency)

l_b_in1 = PWM(Pin(21, Pin.OUT))
l_b_in2 = PWM(Pin(20, Pin.OUT))
l_b_in1.freq(frequency)
l_b_in2.freq(frequency)

drv_left = DRV8833(l_a_in1, l_a_in2, l_b_in1, l_b_in2)
```

```
drv_right = DRV8833(r_a_in1, r_a_in2, r_b_in1, r_b_in2)


MAX_THROTTLE = 0.9  # maximum throttle
var = 0
oth = MAX_THROTTLE

try:
    while True:
        while (var < MAX_THROTTLE):
            var+=0.05
            drv_left.throttle_a(var)
            drv_left.throttle_b(var)
            drv_right.throttle_a(var)
            drv_right.throttle_b(var)

            # small delay to avoid busy looping
            sleep(0.5)
            print("Speed:", var)

        sleep(4.0)

        while (oth > 0):
            oth -= 0.05

            drv_left.throttle_a(oth)
            drv_left.throttle_b(oth)
            drv_right.throttle_a(oth)
            drv_right.throttle_b(oth)
            sleep(0.5)
            print("Speed:", oth)

        var=0
        oth=MAX_THROTTLE

except KeyboardInterrupt:
    drv_right.stop_a()
    drv_right.stop_b()
    drv_left.stop_a()
    drv_left.stop_b()
```

```
drv_right.deinit()
drv_left.deinit()
```

C Delta v4.1 liftoff test

```
# Equipment
* GPC-3030 Analog Dual-channel PSU
* 1g precision scale
* Delta v4.1 prototype

# Preparation
* Set PSU in parallel mode
* Set both current knobs around 1,5A
* Set voltage knob around 2V
* Connect prototype to laptop usb port
    (for current to the pico board)
* Ensure motor_test.py is running on
    the RPI Pico W, with all motors active,
    and with thrust directed upwards,
    thereby pushing the drone down into
    the scale
* Zero the scale, while the prototype
    is placed on it

# Test
* Connect the PSU ground wire to
    the prototype ground wire
* Connect the PSU voltage wire to
    the prototype voltage wire
* Pause the motor_test.py program
* Set voltage knob to 4,5V (NOT HIGHER)
* Start the drv8833_test program again
    -Propellers should start speeding up,
        and slowing down
    -Preliminary readings appear on
        the scale
* Slowly turn both current knobs up
    to 3.2A (max)
* Read the peak thrust value shown on
    the scale during an motor speed-up cycle

# Ending the test
* Pause the motor_test.py program
* Disconnect the PSU voltage wire from the prototype
```

* Disconnect the PSU ground wire from the prototype
* Turn off devices, and store away equipment

D Timelog

Person	Activity	Date	Total
Before timelog creation			
Noa & Samuel	Mails to companies	—	2h
Noa	Repo setup	—	1h 30min
Samuel	Research & design	—	4h
Noa	Test 1	—	2h
Week 5			
Noa	Timelog creation, sent.md added to repo	29/1	1h 10min
Samuel	Research part list + control physics	29/1	1h 10min
Noa & Samuel	Research, maths, milestone 1 work	29/1	4h 32min
Noa	Writing milestone 1 report	29/1	50min
Noa	Writing components list + purchase doc	—	35min
Samuel	Experimentation with IMU (MPU6050)	—	35min
Noa & Samuel	CAD chassis prototype 1	—	2h 45min
Week 6			
Samuel	Motor driver experimentation, chassis print	—	3h 35min
Noa	Propeller testing, reports	5/2	1h 15min
Samuel	Component report	5/2	45min
Noa & Samuel	Design new chassis, printer issues	6/2	2h 10min
Noa & Samuel	Printing v1 chassis, design v2 chassis, report	9/2	2h 5min
Noa & Samuel	Printing v2 chassis, hardware soldering	9/2	2h
Noa & Samuel	Presentations, chassis printing	10/2	4h 10min
Noa	Chassis v2.3 print, report writing	10/2	1h 10min
Week 7			
Noa & Samuel	Soldering on Beta prototype, test code draft	11/2	2h 10min
Noa & Samuel	Finish Beta prototype, soldering and test	12/2	4h 30min
Noa & Samuel	Chassis prototype design Gamma	12/2	30min
Noa	Finish Gamma chassis, documentation	12/2	1h

Continued on next page

Person	Activity	Date	Total
Noa	Print Gamma chassis, slicing	13/2	∼10min
Noa & Samuel	Component research, propellers, ESC & motors	16/2	2h 30min
Noa	Component weighing and chart work	17/2	1h 55min
Noa	Solder Gamma prototype, test	19/2	7h 10min
Samuel	Presentation transcribing	19/2	1h 20min
Noa & Samuel	Midterm presentation writing and preparation	21/2	1h 40min
Week 9			
Noa & Samuel	Hardware research, procurement, project planning	2/3	2h 40min
Noa & Samuel	Report planning	2/3	3h 30min
Noa & Samuel	Writing report introduction	3/3	2h 30min
Noa & Samuel	Writing report introduction (cont.)	3/3	1h
Samuel	Writing code for manoeuvring drone	4/3	2h
Noa & Samuel	Report introduction writing	4/3	55min
Noa	Presentation week 10 work	5/3	30min
Samuel	Report introduction writing	5/3	1h
Noa	Report work	5/3	30min
Samuel	Researching flight controller code	6/3	2h 30min
Noa	Report work, introduction	6/3	1h 5min
Noa	Report work	6/3	30min
Week 10			
Noa	Report introduction writing	8/3	6h 55min
Samuel	Report writing	8/3	3h 30min
Noa & Samuel	Report writing	13/3	5h 5min
Samuel	Report writing, introduction	13/3	2h 55min
Week 11			
Noa & Samuel	Construction of prototype 4, tests 4.1 & 4.2, planning	16/3	4h 35min
Noa & Samuel	Report writing, hardware order	16/3	1h 20min
Noa	Test 4.3 for prototypes 3 & 4	17/3	1h 20min
Samuel	Report writing	17/3	1h
Samuel	Writing software section	17/3	1h
Noa	Prototype soldering, battery research, test 4.2 (liftoff!)	17/3	5h 10min
Samuel	Report work	17/3	2h

Continued on next page

Person	**Activity**	**Date**	**Total**
Noa	Report work	17/3	3h 35min
Noa	Report (wiring diagram)	18/3	50min
Noa & Samuel	Report	18/3	1h 10min
Samuel	Report (software section)	18/3	50min
Noa	Report (diagrams)	18/3	1h
Samuel	Report (software)	18/3	1h
Noa & Samuel	Report	18/3	2h 5min
Noa & Samuel	Report	18/3	1h 25min
Noa	Chassis design and documentation	19/3	2h 25min
Noa	Report-related work	19/3	1h 45min
Week 12			
Samuel	Presentation preparation	20/3	1h 30min
Noa	Prototype 4 preparation	20/3	3h 35min
Noa	Prototype 4 soldering	20/3	3h 50min
Samuel	Report + prototype work	20/3	2h 30min
Samuel	Presentation preparation	20/3	1h
Noa & Samuel	Presentation and report work	21/3	2h 50min
Noa	Report brainstorming	23/3	31min
Noa	Report-related work	24/3	2h 32min
Samuel	Report	24/3	2h 2min
Noa & Samuel	Thesis planning	24/3	1h 30min
Noa & Samuel	Report	24/3	1h 20min
Samuel	Report	24/3	1h 10min
Noa & Samuel	Report	24/3	1h 45min
Noa	Report (results)	25/3	1h 20min
Samuel	Report (results)	25/3	40min
Noa	Report (results)	26/3	1h
Samuel	Software development	26/3	1h
Samuel	Software development	26/3	1h 42min
Noa	Report	26/3	2h 22min
Noa	Report	26/3	2h
Samuel	Report	26/3	2h
Noa	Report results	26/3	3h 3min
Noa	Report results	27/3	1h 20min
Noa	Report results	27/3	5h 39min
Noa	Report finish	27/3	34min
Samuel	Report finish	27/3	5h 36min
Noa	**∼111h 40min**		
Samuel	**∼95h 37min**		

E Component list for delta prototype

Component name	Amnt	Buy-link	Cost/pcs	Total cost
Raspberry Pi Pico W	1 pcs		88,00 SEK	88,00 SEK
DRV8833	4 pcs		12,79 SEK	51,16 SEK
DC motors (set of 4)	1 set		54,40 SEK	54,40 SEK
Chassis (PLA filament)	$\sim$6 g		199,00 SEK	1,19 SEK
Tools, wiring, solder	enough		varies	–
Optional, recommended				
MPU6050	1 pcs		65,00 SEK	65,00 SEK
Battery (untested)	1+ pcs		102,00 SEK	102,00 SEK

Table 5.5: Table of materials for prototype delta series